

Livrable 2 — Dossier Technique

1) Architecture globale

Le module est organisé en quatre blocs:

1. Bloc de données continu

- Les 10 000 drones sont alloués d'un seul coup avec `malloc` dans un espace contigu de type:

```
struct Drone { int id; float x; float y; float z; };
```
- L'accès se fait uniquement par arithmétique de pointeurs: `(*(swarm + i)).x`.

2. Tri fusion personnalisé (sans qsort)

- Deux tableaux de pointeurs vers drones sont construits:
 - un trié par `x`
 - un trié par `y`
- Le tri est un **Merge Sort** codé manuellement, sans indexation par crochets.

3. Noyau Divide and Conquer

- Découpage en deux moitiés selon l'axe `x`.
- Résolution récursive à gauche et à droite.
- Fusion logique via une bande centrale (strip) pour détecter une paire transversale plus proche.

4. Sortie sécurité

- Retour de la paire de drones minimale `(A, B)` et de `distance2`.
- Cette information peut déclencher la manœuvre d'évitement.

2) Pourquoi l'arithmétique de pointeurs est adaptée au temps réel

- **Accès mémoire prévisible:** le bloc continu améliore la localité cache (L1/L2), ce qui réduit les latences mémoire.

- **Moins de surcoût logiciel:** les lectures/écritures via `*(ptr + i)` évitent des couches d'abstraction et restent proches des instructions machine.
- **Coût constant par accès:** chaque drone est atteint par une adresse de base + décalage, ce qui se mappe directement à l'adressage CPU.
- **Traitement massif en 1 ms:** sur 10 000 drones, réduire les défauts de cache et garder des parcours séquentiels apporte un gain tangible en cycles.

3) Mécanique mémoire (lecture/écriture sans crochets)

Exemple de navigation dans le tas:

- Adresse du i-ème drone: `(swarm + i)`
- Déréférencement structuré: `*(swarm + i).y`
- Écriture: `*(swarm + i).z = valeur;`
- Lecture: `val = *(swarm + i).x;`

Cette forme démontre explicitement le décalage mémoire en octets géré par le compilateur selon `sizeof(struct Drone)`.

4) Robustesse de l'implémentation

- Vérification des retours `malloc`.
- Libération systématique des blocs alloués.
- Chemins d'erreur avec sortie explicite.
- Tri et calcul de distance réalisés avec des types flottants adaptés aux coordonnées 3D.